

Absolutely. Once you understand commits, branches, merge, rebase, fixup, and cherry-pick, the next level is less about memorizing commands and more about **recovering safely, inspecting history, and making precise changes**.

These are the Git features I'd prioritize.

1. `git reflog`: your emergency recovery tool

`reflog` records where your local branch pointers and `HEAD` recently moved.

Use it when you:

- accidentally reset a branch
- lose commits after a rebase
- delete a branch
- check out the wrong commit
- think your work disappeared

Bash

```
git reflog
```

Example:

```
a82cd11 HEAD@{0}: reset: moving to origin/main  
93fe120 HEAD@{1}: commit: Implement notification handling
```

Recover the lost commit:

Bash

```
git switch -c recovery/notification 93fe120
```

Or restore the current branch:

Bash

```
git reset --hard 93fe120
```

Use the second command carefully because it replaces the working tree.

Developer survival rule: before panicking, run `git reflog`.

2. `git restore`: undo changes precisely

Restore an unstaged file:

Bash

```
git restore path/to/file
```

Unstage a file without deleting its changes:

Bash

```
git restore --staged path/to/file
```

Restore a file from another commit:

Bash

```
git restore --source=<commit> path/to/file
```

For example:

Bash

```
git restore --source=origin/main config/app.yml
```

Restore both staged and working-tree content:

Bash

```
git restore --source=HEAD --staged --worktree path/to/file
```

Be cautious: restoring the working tree discards local modifications.

3. Interactive staging with `git add -p`

This is one of the most valuable everyday commands.

Bash

```
git add -p
```

Git shows each change block and asks whether to stage it.

Common choices:

```
y = stage this hunk
n = don't stage it
s = split it into smaller hunks
e = manually edit the hunk
q = quit
```

Suppose one file contains:

- a bug fix
- debugging logs
- unrelated refactoring

Instead of committing the whole file, stage only the bug fix:

Bash

```
git add -p
git commit -m "Fix session restoration"
```

Then handle the remaining work separately.

This helps create logical commits even when your working directory is messy.

Related commands:

Bash

```
git restore -p .
git reset -p
```

4. Interactive rebase beyond autosquash

You already know:

Bash

```
git rebase -i origin/main
```

The full set of useful actions includes:

pick	keep the commit
reword	change its message
edit	pause and modify it
squash	combine it with the previous commit
fixup	combine and discard its message
drop	remove the commit

Example:

pick	a123abc	Add API client
reword	b234bcd	Add tests
edit	c345cde	Add retry handling
drop	d456def	Temporary debugging

When Git pauses at an `edit` commit:

Bash

```
# Modify files
git add .
git commit --amend
git rebase --continue
```

To split one commit into multiple commits:

Bash

```
git rebase -i origin/main
```

Mark the commit as `edit`, then:

Bash

```
git reset HEAD^
git add -p
git commit -m "First logical change"

git add -p
git commit -m "Second logical change"

git rebase --continue
```

5. `git commit --amend`

Correct the most recent commit:

Bash

```
git add <files>
git commit --amend
```

Change only its message:

Bash

```
git commit --amend -m "Better commit message"
```

Amend without changing the message:

Bash

```
git commit --amend --no-edit
```

This rewrites the commit hash. It is ideal before sharing the commit, but be deliberate when it has already been pushed.

6. `git stash` with useful options

Basic stash:

Bash

```
git stash push -m "WIP notification tests"
```

List stashes:

Bash

```
git stash list
```

Inspect one:

Bash

```
git stash show -p stash@{0}
```

Apply while keeping the stash:

Bash

```
git stash apply stash@{0}
```

Apply and remove it:

Bash

```
git stash pop
```

Include untracked files:

Bash

```
git stash push -u -m "WIP"
```

Stash only selected changes:

Bash

```
git stash push -p
```

Create a branch directly from a stash:

Bash

```
git stash branch feature/notification-fix stash@{0}
```

This last command is extremely useful when stashed work becomes a real task.

Do not let `stash` become a mysterious attic full of six-month-old boxes. Prefer temporary commits for important work.

7. `git bisect` : find which commit introduced a bug

`git bisect` performs a binary search through commit history.

Start:

Bash

```
git bisect start
```

Mark the current broken commit:

Bash

```
git bisect bad
```

Mark a known working commit:

Bash

```
git bisect good <commit>
```

Git checks out a commit in the middle. Test it and report:

Bash

```
git bisect good
```

or:

Bash

```
git bisect bad
```

Eventually Git identifies the first bad commit.

Finish:

Bash

```
git bisect reset
```

You can automate it:

Bash

```
git bisect start HEAD <known-good-commit>
git bisect run npm test
```

The test command should return:

- 0 for good
- a nonzero value for bad

This is incredibly powerful for regressions in large repositories.

8. Better history inspection

A useful default graph:

Bash

```
git log --graph --decorate --oneline --all
```

Show commits affecting one file:

Bash

```
git log --oneline -- path/to/file
```

Follow a renamed file:

Bash

```
git log --follow -- path/to/file
```

Show the patch for each commit:

Bash

```
git log -p -- path/to/file
```

Search commit messages:

Bash

```
git log --grep="notification"
```

Search for commits that added or removed a string:

Bash

```
git log -S "restoreSession"
```

Search for commits whose diff matches a pattern:

Bash

```
git log -G "restoreSession.*timeout"
```

The distinction:

- `-s` finds changes in how many times an exact string appears.
 - `-G` finds diffs matching a regular expression.
-

9. `git blame`, used responsibly

Find who last changed each line:

Bash

```
git blame path/to/file
```

Limit to specific lines:

Bash

```
git blame -L 50,80 path/to/file
```

Inspect the corresponding commit:

Bash

```
git show <commit>
```

`git blame` is best used to find **context**, not a culprit. The name has terrible workplace energy; mentally rename it `git archaeology`.

10. Comparing branches correctly

Two-dot comparison:

Bash

```
git diff origin/main..HEAD
```

This compares the current snapshots.

Three-dot comparison:

Bash

```
git diff origin/main...HEAD
```

This compares your branch with its merge base against `origin/main`. It usually represents the changes introduced by your PR.

Show commits unique to your branch:

Bash

```
git log origin/main..HEAD
```

See commits unique to either side:

Bash

```
git log --left-right --graph origin/main...HEAD
```

For PR review, these are particularly useful:

Bash

```
git diff origin/main...HEAD  
git log --oneline origin/main..HEAD
```

11. `git switch` and detached HEAD

Inspect an old commit safely:

Bash

```
git switch --detach <commit>
```

You are now in detached HEAD mode. You can examine or test the repository without moving a branch.

To keep work created there:

Bash

```
git switch -c experiment/from-old-commit
```

To return:

Bash

```
git switch -
```

The dash switches back to the previous branch.

12. `git worktree` : use multiple branches simultaneously

This is excellent when you need to:

- work on one feature
- review another PR
- quickly fix production
- run tests on two branches

Create another working directory:

Bash

```
git worktree add ../project-hotfix -b hotfix/session origin/main
```

Now you have:

project/	current feature branch
project-hotfix/	hotfix/session branch

List worktrees:

Bash

```
git worktree list
```

Remove one:

Bash

```
git worktree remove ../project-hotfix
```

This is often cleaner than constantly stashing and switching branches.

13. `git rerere` : remember conflict resolutions

`rerere` means “reuse recorded resolution.”

Enable it:

Bash

```
git config --global rerere.enabled true
```

When you resolve a conflict, Git records how you resolved it. If the same conflict occurs again during another rebase or merge, Git can reuse the resolution.

This is valuable when repeatedly rebasing a long-running feature branch.

Inspect recorded resolutions:

Bash

```
git rerere status
git rerere diff
```

It sounds obscure, but people dealing with recurring conflicts become suspiciously passionate about it.

14. `git range-diff`: review a rebased branch

After rebasing or autosquashing, commit hashes change. A normal diff may not clearly show how the series changed.

Suppose the previous version of your branch is tagged or saved:

Bash

```
git branch backup/before-rebase
git rebase -i origin/main
```

Compare the old and new commit series:

Bash

```
git range-diff origin/main...backup/before-rebase origin/main...HEAD
```

This helps answer:

Did my rebase only reorganize commits, or did I accidentally change the code?

It is especially useful for reviewers after a force-push.

15. `git revert`: safely undo shared history

Undo a published commit without rewriting history:

Bash

```
git revert <commit>
```

This creates a new commit containing the inverse change.

Revert several commits without immediately committing:

Bash

```
git revert --no-commit <commit-1> <commit-2>
git commit -m "Revert unstable notification changes"
```

Reverting a merge commit requires selecting the mainline parent:

Bash

```
git revert -m 1 <merge-commit>
```

That operation deserves extra care because later re-merging the same branch can become confusing.

General rule:

- Unpublished local history: `reset` , rebase, amend.
- Shared published history: `revert` .

16. Safer force-pushing

After rebase:

Bash

```
git push --force-with-lease
```

An even stricter workflow can include the expected remote commit:

Bash

```
git push --force-with-lease=refs/heads/my-branch:<expected-hash>
```

Never make plain `--force` your default.

You can protect yourself:

Bash

```
git config --global push.useForceIfIncludes true
```

And configure automatic remote tracking:

Bash

```
git config --global push.autoSetupRemote true
```

17. Useful configuration

Automatically autosquash:

Bash

```
git config --global rebase.autosquash true
```

Automatically prune deleted remote branches during fetch:

Bash

```
git config --global fetch.prune true
```

Use rebase for pulls:

Bash

```
git config --global pull.rebase true
```

Or require fast-forward-only pulls:

Bash

```
git config --global pull.ff only
```

Show conflict base, ours, and theirs:

Bash

```
git config --global merge.conflictstyle zdiff3
```

Enable remembered resolutions:

Bash

```
git config --global rerere.enabled true
```

A reasonable setup for personal feature branches is:

Bash

```
git config --global fetch.prune true
git config --global rebase.autosquash true
git config --global rerere.enabled true
git config --global merge.conflictstyle zdiff3
git config --global push.autoSetupRemote true
```

Choose `pull.rebase` or `pull.ff only` according to your team workflow rather than copying it blindly.

What I'd learn first

Prioritize these in order:

1. `git reflog`
2. `git add -p`
3. `git restore`
4. advanced interactive rebase
5. `git bisect`
6. `git worktree`
7. `git range-diff`
8. `git rerere`

Those give you the biggest jump from “I know Git commands” to “I can confidently recover and control Git history.”